

Working with LOBs

1. What is a LOB?

LOB (Large Object) is a data type used to store large amounts of data in databases. It can be divided into:

- **BLOB (Binary Large Object):**
 - Stores binary data such as images, audio, and video files.
 - Represented as `byte[]` in Java.
- **CLOB (Character Large Object):**
 - Stores large text data like documents, XML, or HTML.
 - Represented as `char[]` or `String` in Java.

2. How to Read Objects (BLOB and CLOB)

Reading a BLOB

```
FileInputStream fis = new  
FileInputStream("D:\\IO\\Java.JPG");  
image = new byte[fis.available()];  
fis.read(image);
```

- **FileInputStream:**
 - Used to read binary data from a file.
 - `new FileInputStream("D:\\IO\\Java.JPG")`: Opens a stream to read the file `Java.JPG`.
- **fis.available():**
 - Returns the number of bytes available in the file for reading.
 - Allocates a `byte[]` array of the required size.
- **fis.read(image):**
 - Reads the file's content into the `byte[]` array.

Reading a CLOB

```
File file = new File("D:\\IO\\PersonalInfo.txt");
reader = new FileReader(file);
textFile = new char[(int) file.length()];
reader.read(textFile);
```

- **File:**
 - Represents the text file `PersonalInfo.txt`.
 - `new File("D:\\IO\\PersonalInfo.txt")`: Points to the file location.
- **FileReader:**
 - Used to read character data from a file.
 - `new FileReader(file)`: Opens the file for reading.
- **file.length():**
 - Returns the file size in bytes, cast to `int` to create a `char[]`.
- **reader.read(textFile):**
 - Reads the file's content into the `char[]` array.

3. Writing Retrieved Data to Local System

Writing BLOB Data

```
FileOutputStream fos = new FileOutputStream("Java.JPG");
fos.write(studentInfo.getImage());
```

- **FileOutputStream:**
 - Used to write binary data to a file.
 - `new FileOutputStream("Java.JPG")`: Opens a stream to write to the file `Java.JPG`.
- **fos.write(studentInfo.getImage()):**
 - Writes the binary data (image) from the `studentInfo` object into the file.

Writing CLOB Data

```
FileWriter writer = new FileWriter("PersonalInfo.txt");  
writer.write(studentInfo.getTextFile());
```

- **FileWriter:**
 - Used to write character data to a file.
 - `new FileWriter("PersonalInfo.txt")`: Opens a stream to write to the file `PersonalInfo.txt`.
- **`writer.write(studentInfo.getTextFile())`:**
 - Writes the character data (text) from the `studentInfo` object into the file.

4. Use of Annotations in the Code

@Lob

- Indicates that the field should be mapped as a large object in the database.
- Automatically maps:
 - `byte[]` to a BLOB column.
 - `char[]` or `String` to a CLOB column.

@Column(length=100000)

- Sets a column-specific attribute in the database.
- **`length=100000`:**
 - Specifies the maximum size of the column for storing the BLOB.

Entity Class Example:

```
@Lob  
@Column(length = 100000)  
private byte[] image;  
  
@Lob  
private char[] textFile;
```

- **image:**
 - Stores binary data as a BLOB.
 - Limited to a maximum of 100,000 bytes.
- **textFile:**
 - Stores character data as a CLOB.

5. Step-by-Step Summary

1. Reading BLOB and CLOB:

- Use `FileInputStream` and `FileReader` to read the data from files.

2. Writing Data Locally:

- Use `FileOutputStream` and `FileWriter` to save the data to files.

3. Database Mapping:

- Use `@Lob` for large objects and configure with `@Column` for custom sizes.

4. Stream Management:

- Always close streams after usage to avoid resource leaks.